

THADOMAL SHAHANI ENGINEERING COLLEGE

University of Mumbai

Department of Computer Engineering

DATA WAREHOUSING AND MINING LABORATORY

Lab Code 2115120 | Semester V | Academic Year 2026-27

HANDWRITTEN WRITEUPS — ALL EXPERIMENTS

Experiments 1 to 8 — aim, theory, procedure, results and conclusion for the lab journal

CASE STUDY

Predicting Trends and Uncovering Patterns in Bitcoin Price Movement Using Data Mining Techniques

Retail investors trade Bitcoin on hype, influencer opinion and panic rather than on evidence. Bitcoin trades 24x7 and has produced an uninterrupted daily OHLC series since July 2010 that covers several documented boom-bust cycles. This project builds a data mart over 5,684 daily records (17 July 2010 to 27 March 2026, 20 attributes) and mines it for patterns in trend and volatility — comparing behaviour across the five block-reward (halving) eras, discovering recurring market regimes, and benchmarking how far next-day direction can honestly be predicted.

Domain	Financial Technology (FinTech) — cryptocurrency market analytics
Dataset	btc_flat_final_2.csv — 5,684 daily records, 20 attributes, 1.10 MB
Period covered	17 July 2010 to 27 March 2026 (15 years 8 months)
Tools used	PostgreSQL 16, Python 3 (pandas, NumPy, matplotlib, seaborn, SciPy), Microsoft Excel, Jupyter Notebook
Techniques applied	Dimensional modelling, ETL, OLAP, preprocessing, classification, clustering, association rule mining

SUBMITTED BY

Name	Roll No.	Class
Divvy Kansara	2403080	TE Computer Engineering
Jai Kukreja	2403090	TE Computer Engineering

SUBJECT IN-CHARGE

Dr. Ujwala Bharambe

Date of submission: 13 September 2026

INDEX

Exp	Title	CO	Contents
1	Case Study, Problem Statement and Dimensional Modelling	LO1	Data warehouse theory, fact and dimension design, star vs snowflake
2	Implementation of Dimension and Fact Tables in PostgreSQL	LO1	ETL theory, surrogate keys, constraints, ER diagram of the implemented schema
3	Implementation of OLAP Operations	LO2	OLAP vs OLTP, the data cube, the five OLAP operations with results
4	Data Preprocessing and Exploration	LO3	Preprocessing theory, missing-value strategies, normalization and discretization methods
5	Naive Bayesian and Classification Tree Algorithms	LO4	Bayes theorem, entropy and information gain, confusion matrix and metrics
6	Implementation of Clustering Algorithm (K-Means)	LO4	Clustering theory, K-Means algorithm and flowchart, full solved numerical example
7	Implementation of Hierarchical Clustering	LO4	AGNES and DIANA, linkage criteria, Lance-Williams formula, dendrogram
8	Implementation of Association Rule Mining (Apriori)	LO4	Support, confidence, lift, the Apriori property, the level-wise algorithm

EXPERIMENT NO. 1

Case Study, Problem Statement and Dimensional Modelling

Star and snowflake schema design for the Bitcoin price data mart

1. Aim

To select a case study for building a data warehouse / data mart, write a detailed problem statement, and design the dimensional model by constructing the star schema and the snowflake schema.

2. Theory

2.1 Data warehouse and data mart

A **data warehouse** is a subject-oriented, integrated, time-variant and non-volatile collection of data that supports management's decision-making process (Inmon). A **data mart** is a smaller, subject-specific slice of a warehouse built for one business area — here, Bitcoin price analytics.

Four key properties:

- **Subject-oriented** — organised around a subject (price behaviour), not around daily operations.
- **Integrated** — data from different sources is made consistent in naming, units and encoding.
- **Time-variant** — every record is tied to a point in time; history is retained, not overwritten.
- **Non-volatile** — data is loaded and read, never updated transactionally by end users.

2.2 Dimensional modelling

Dimensional modelling designs the warehouse around **facts** and **dimensions**.

- **Fact table** — holds the numeric measures of a business process plus foreign keys to the dimensions. Measures are *additive* (can be summed across all dimensions), *semi-additive* (summable across some dimensions only) or *non-additive* (ratios, percentages).
- **Dimension table** — holds the descriptive attributes ("who, what, where, when") used to filter and group the facts. Dimensions are small, wide and textual.
- **Grain** — the meaning of a single fact row. It is fixed before anything else is designed.
- **Surrogate key** — a system-generated integer primary key for each dimension row, independent of the source system's business key.

2.3 Schema types

Schema	Structure	Characteristics
Star	One fact table joined to denormalised dimension tables	Simple, few joins, fast queries, some redundancy; dimensions in 2NF
Snowflake	Dimensions normalised into hierarchy tables	Less redundancy, more joins, slower aggregation; dimensions in 3NF
Fact constellation (galaxy)	Several fact tables sharing conformed dimensions	Models multiple business processes; most complex

3. Problem Statement (case study)

Retail investors trade Bitcoin on hype and panic rather than on evidence. Bitcoin trades 24x7 and has produced an uninterrupted daily OHLC series since July 2010 covering several boom-bust cycles. This project builds a data mart over 5,684 daily records (17 Jul 2010 – 27 Mar 2026, 20 attributes) and mines it for patterns in trend and volatility, comparing behaviour across the five block-reward (halving) eras.

Business requirements: volatility and risk profiling; halving-era comparison; trend and momentum analysis; market-regime discovery; a benchmarked next-day direction classifier; and roll-up reporting from daily prices to monthly, quarterly, yearly and era level.

4. Dimensional Model

Grain: one row per asset per calendar day.

Component	Contents
FACT_BTC_DAILY	FKs: date_key, asset_key, era_key, state_key. Measures: open, high, low, close, daily_change, daily_pct_change, daily_range, ma_7, ma_30, ma_200, volatility_30d, ema_12, ema_26, macd, macd_signal, rsi_14. Degenerate: days_since_halving
DIM_DATE	full_date, day_of_month, day_of_week, week_of_year, month_no, month_name, quarter, year, is_month_end
DIM_ASSET	ticker, asset_name, asset_type
DIM_HALVING_ERA	era_name, block_reward_btc, era_start_date, era_end_date
DIM_MARKET_STATE	direction (Up/Down/Flat), volatility_band (Low/Medium/High), rsi_zone (Oversold/Neutral/Overbought/Unknown) — 36 combinations

Fig. 1 Star schema — Bitcoin daily price data mart

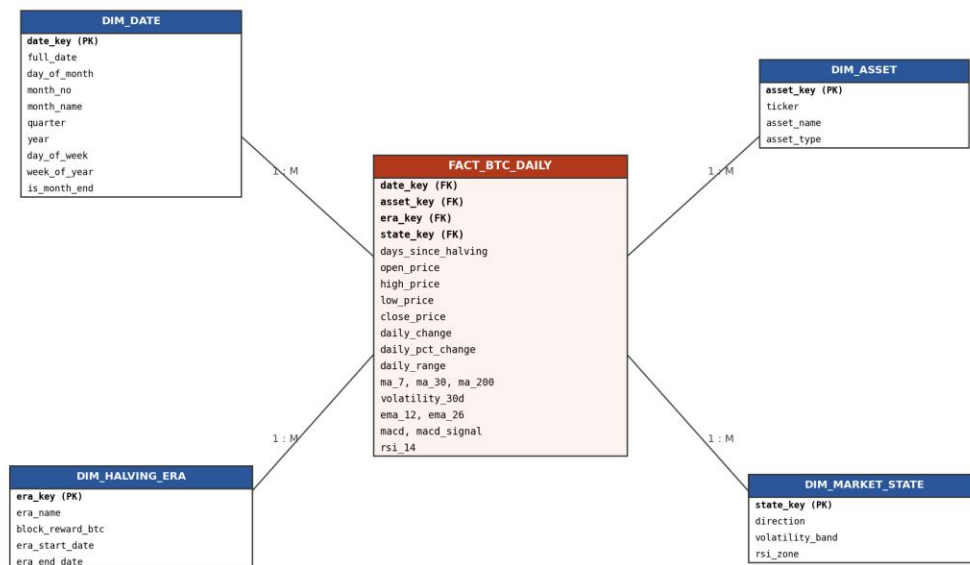
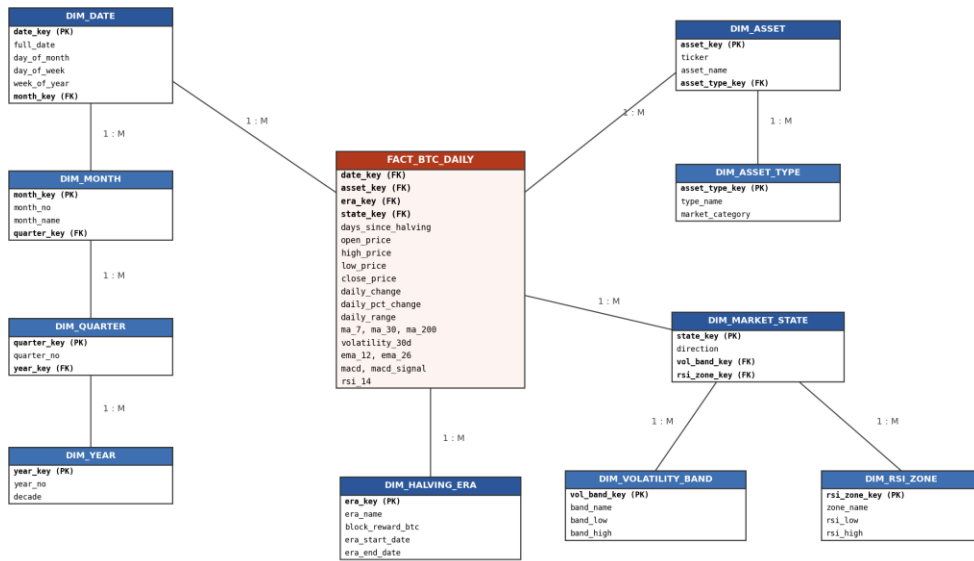


Fig. 2 Snowflake schema — date, asset and market-state dimensions normalised



Star vs snowflake for this data mart

Criterion	Star	Snowflake
Dimension tables	4	10
Joins for a yearly roll-up	1	4
Query speed	Faster	Slower
Redundancy	Higher	Lower
Normal form of dimensions	2NF	3NF

Chosen: star schema — the mart is read-heavy, the dimensions are tiny (5,733 dates, 5 eras, 36 market states, 1 asset), so query simplicity outweighs the storage the snowflake would save.

5. Conclusion

A Bitcoin price case study was selected and its problem statement and business requirements were written. The dataset of 5,684 daily records with 20 attributes was described and a data dictionary prepared. A dimensional model was designed at a grain of one asset-day with one fact table and four dimensions, and was drawn as both a star schema and a snowflake schema; the star schema was selected for implementation in PostgreSQL in Experiment 2.

EXPERIMENT NO. 2

Implementation of Dimension and Fact Tables in PostgreSQL

ETL from the raw CSV into the Bitcoin star-schema data mart

1. Aim

To implement the dimensional model of Experiment 1 in PostgreSQL — creating the database, the staging table, the dimension tables and the fact table, loading the data through an ETL process and verifying it.

2. Theory

2.1 The three-layer load

A warehouse is loaded in three stages, known as **ETL**:

- **Extract** — read the source data as it is. It lands in a **staging table** that mirrors the source: same columns, no keys, no constraints, so that a malformed row cannot block the load.
- **Transform** — clean, deduplicate, derive attributes and exchange business keys for surrogate keys.
- **Load** — insert the transformed rows into the dimension tables first and the fact table last, because the fact rows need the dimension keys to exist.

2.2 Keys

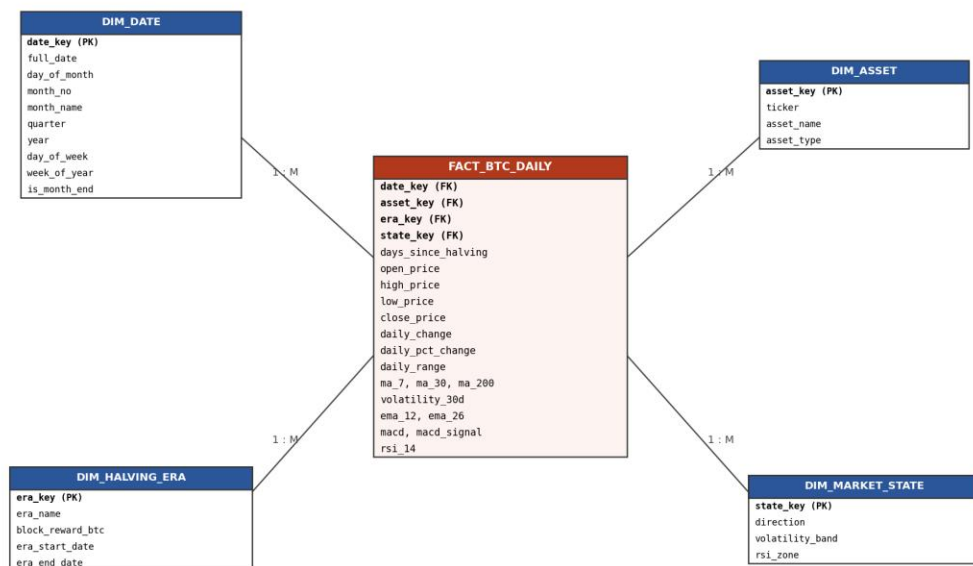
- **Business (natural) key** — the identifier used by the source system, e.g. the trading date.
- **Surrogate key** — a meaningless integer generated by the warehouse and used as the dimension's primary key. It insulates the warehouse from source-system changes, keeps joins narrow and fast, and allows slowly changing dimensions to store several versions of one business entity.
- **Smart key** — a surrogate key with a readable pattern, e.g. `date_key = yyymmdd = 20240101`.
- **Degenerate dimension** — a dimension attribute with no other attributes of its own, so it is kept in the fact row itself (here: `days_since_halving`).

2.3 Constraints used

Constraint	Where used	Purpose
PRIMARY KEY	every dimension, fact_key	row identity
FOREIGN KEY	4 keys in the fact table	referential integrity of the star
UNIQUE	(date_key, asset_key)	enforces the declared grain
CHECK	prices > 0; allowed state values	domain integrity
INDEX	every foreign key	fast star joins for OLAP

3. ER Diagram of the Implemented Schema

Fig. 1 Star schema — Bitcoin daily price data mart



One-to-many from each dimension to the fact table; the fact table carries the four foreign keys, the degenerate dimension and all measures.

4. Procedure

1. **Create the database** — CREATE DATABASE btc_dw; then connect with \c btc_dw.
2. **Create the tables** — staging first, then the four dimensions, then the fact table (it references them), followed by the indexes.
3. **Load staging** — \copy staging_btc_daily FROM 'btc_flat_final_2.csv' WITH (FORMAT csv, HEADER true, NULL '') → 5,684 rows.
4. **Populate the dimensions** — DIM_DATE generated with generate_series over the full date range (a date dimension must be gap-free); DIM_ASSET, DIM_HALVING_ERA deduplicated from staging; DIM_MARKET_STATE built as the cross product of its three attributes.
5. **Populate the fact table** — join staging to each dimension on its business key, take the surrogate key, remove the duplicate date with DISTINCT ON, and insert.
6. **Verify** — row counts, orphan checks, grain check, business aggregation.

5. Results

Table	Rows	Note
staging_btc_daily	5,684	raw CSV
dim_date	5,733	every day 17-Jul-2010 to 27-Mar-2026
dim_asset	1	BTC
dim_halving_era	5	Genesis to Post-Halving 4

Table	Rows	Note
dim_market_state	36	3 x 3 x 4 attribute combinations
fact_btc_daily	5,683	one duplicate date removed

Integrity: orphan rows against all four dimensions = 0; duplicate grain rows = 0; 50 calendar days legitimately carry no fact row (the gaps present in the source).

First business result: average absolute daily move per era — 4.611%, 3.433%, 2.724%, 2.210%, 1.745% from the Genesis era to Post-Halving 4 — a monotone fall, i.e. the market has become calmer with every halving.

6. Conclusion

The star schema was implemented in PostgreSQL with surrogate keys, foreign keys, a unique grain constraint, check constraints and indexes. 5,684 staging rows were transformed into 5,683 fact rows joined to four populated dimensions, with zero referential-integrity violations. The data mart is now queryable and is used for the OLAP operations of Experiment 3.

EXPERIMENT NO. 3

Implementation of OLAP Operations

Slice, dice, roll-up, drill-down and pivot in PostgreSQL and in Excel

1. Aim

To implement the OLAP operations — slice, dice, roll-up, drill-down and pivot — in SQL on the Bitcoin data mart, and to perform the same operations using a Pivot Table in Excel.

2. Theory

2.1 OLAP

OLAP (On-Line Analytical Processing) is the category of software that lets analysts query a multidimensional view of data interactively. It is contrasted with **OLTP**:

Property	OLTP	OLAP
Purpose	run the business (day-to-day transactions)	analyse the business (decision support)
Operations	short INSERT / UPDATE / DELETE	long, complex read-only queries
Data	current, detailed, normalised	historical, summarised, dimensional
Users	clerks, applications	analysts, managers

2.2 The data cube

OLAP data is modelled as a **cube**: the dimensions are the edges and the measures are the cells. This project's cube has four dimensions — time, asset, halving era and market state — and the daily price measures as cells. Each dimension has a **hierarchy** (day → month → quarter → year), and OLAP operations are movements along or across those hierarchies.

2.3 The five OLAP operations

Operation	Definition	Example in this experiment
Roll-up	climb a hierarchy / reduce dimensions; the data becomes more summarised	daily prices summarised to yearly averages
Drill-down	the reverse of roll-up: descend to finer detail	halving era expanded into its months
Slice	fix ONE dimension to a single value, giving a sub-cube of one lower dimension	direction = 'Down'
Dice	select a range/set of values on TWO or more dimensions, giving a smaller cube	year 2020-2022 AND volatility 'High' AND direction 'Up'
Pivot (rotate)	rotate the cube so a dimension's members become columns	year (rows) x direction (columns)

Related SQL operators: **ROLLUP** gives the sub-totals along one hierarchy plus a grand total; **CUBE** gives every combination of the grouping columns (2^n groupings); **GROUPING SETS** lets the exact set of groupings be listed explicitly.

3. Queries Used (short form)

```
-- ROLL-UP : daily facts summarised to the year
SELECT d.year, COUNT(*), AVG(f.close_price)
FROM fact_btc_daily f JOIN dim_date d ON d.date_key = f.date_key
GROUP BY d.year ORDER BY d.year;

-- DRILL-DOWN : one era expanded into months
... JOIN dim_halving_era e ... WHERE e.era_name = 'Post-Halving 4 (3.125 BTC/block)'
GROUP BY d.year, d.month_no, d.month_name;

-- SLICE : one dimension fixed
... JOIN dim_market_state m ... WHERE m.direction = 'Down' GROUP BY d.year;

-- DICE : several dimensions restricted
... WHERE d.year BETWEEN 2020 AND 2022 AND m.volatility_band = 'High' AND m.direction = 'Up';

-- PIVOT : members of a dimension become columns
SELECT d.year,
       COUNT(*) FILTER (WHERE m.direction = 'Up') AS up_days,
       COUNT(*) FILTER (WHERE m.direction = 'Flat') AS flat_days,
       COUNT(*) FILTER (WHERE m.direction = 'Down') AS down_days
FROM ... GROUP BY d.year;
```

4. Results

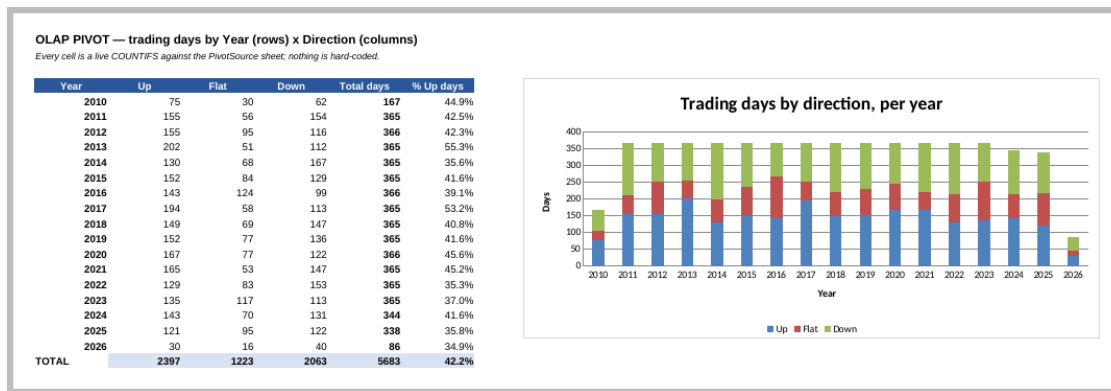
Pivot — year x direction (extract):

Year	Up	Flat	Down	Total	% Up
2013	202	51	112	365	55.3%
2021	165	53	147	365	45.2%
2022	129	83	153	365	35.3%
2025	121	95	122	338	35.8%
All	2397	1223	2063	5683	42.2%

Pivot — halving era x volatility band (average absolute daily move %):

Halving era	Low	Medium	High	High-vol days
Genesis (50 BTC)	0.69	2.27	9.30	334
Post-Halving 1 (25)	0.72	2.20	9.63	309
Post-Halving 2 (12.5)	0.63	2.35	6.72	297
Post-Halving 3 (6.25)	0.66	2.23	6.38	175
Post-Halving 4 (3.125)	0.73	2.34	5.68	28

The Excel PivotTable, built on the same exported rows with independent COUNTIFS / AVERAGEIFS formulas, reproduces every one of these figures — a cross-check on both implementations.



Excel pivot: trading days by year and direction, with the stacked-column PivotChart.

5. Conclusion

All five OLAP operations were implemented in SQL on the Bitcoin star schema and reproduced with a Pivot Table in Excel; the two agree on every value checked. The analysis shows that volatility has fallen in every successive halving era (average move on high-volatility days: 9.30% in the Genesis era against 5.68% in Post-Halving 4, with high-volatility days falling from 334 to 28), and that up days slightly outnumber down days overall (2,397 against 2,063 out of 5,683).

EXPERIMENT NO. 4

Data Preprocessing and Exploration

Descriptive analysis, null-value handling, normalization and discretization

1. Aim

To implement data preprocessing on the Bitcoin dataset: descriptive analysis, handling of null values, data normalization and data discretization.

2. Theory

2.1 Why preprocess

Real data is **incomplete** (missing values), **noisy** (errors and outliers) and **inconsistent** (conflicting or duplicated records). Data mining on such data gives unreliable results — "garbage in, garbage out". Preprocessing has four major tasks:

Task	What it does
Data cleaning	fill missing values, smooth noise, remove duplicates, correct inconsistencies
Data integration	merge data from multiple sources, resolve naming and unit conflicts
Data transformation	normalization, aggregation, generalisation, attribute construction
Data reduction	dimensionality reduction, numerosity reduction, discretization

2.2 Descriptive analysis

Summarises the data numerically before any model is built:

- **Central tendency** — mean, median, mode.
- **Dispersion** — range, quartiles, IQR, variance, standard deviation.
- **Shape** — **skewness** (asymmetry; 0 = symmetric, >0 = right tail) and **kurtosis** (tail weight; >3 excess = heavy tails).
- **Five-number summary** — min, Q1, median, Q3, max, drawn as a boxplot.

2.3 Handling missing values

Method	When to use	Risk
Ignore the tuple	few rows, missing target	data loss
Fill manually	very small datasets	infeasible at scale
Global constant	"Unknown" category	creates a fake class
Mean / median	numeric, MCAR	shrinks variance
Forward / backward fill	time series	cannot fix a leading gap
Interpolation	continuous time series	assumes smoothness
Model-based (regression, KNN)	strong correlations exist	costly, can over-fit

Missingness is **structural** (the value cannot exist — a 200-day average on day 5) or **at random** (the value exists but was not recorded). Structural nulls should be removed, not imputed.

2.4 Normalization

Min-max : $v' = (v - \min) / (\max - \min) \times (\text{new_max} - \text{new_min}) + \text{new_min}$

Z-score : $v' = (v - \mu) / \sigma$

Decimal scale: $v' = v / 10^j$ where j is the smallest integer with $\max(|v'|) < 1$

Normalization puts attributes on a comparable scale so that distance-based algorithms (K-Means, KNN, hierarchical clustering) are not dominated by whichever attribute has the widest numeric range. It changes the axis, not the shape of the distribution.

2.5 Discretization

Converting a continuous attribute into intervals:

- **Equal-width** — k intervals of equal size, $\text{width} = (\max - \min) / k$. Simple; fails on skewed data.
- **Equal-frequency (quantile)** — each interval holds the same number of records. Robust to skew.
- **Domain / concept-based** — cut-points from the meaning of the attribute (RSI 30 and 70).
- **Entropy-based, ChiMerge, clustering-based** — supervised and statistical alternatives.

3. Procedure and Results

Part A — cleaning. The existing script was reviewed and corrected. It crashed on its own configured input (*ValueError: cannot reindex on an axis with duplicate labels*) because the duplicate date 2010-08-02 is reported but never removed. Four further defects were fixed: the halving-day counter was being reset (666 rows changed), interpolated rows were unflagged, the burn-in was cut using a null count instead of a position, and OHLC consistency was not re-checked after interpolation.

Stage	Rows	Nulls
Raw	5,684	279
Duplicate date removed	5,683	279
50 calendar gaps filled	5,733	279
199 burn-in rows dropped	5,534	0

Descriptive analysis. close: skew 1.595 (right-skewed); daily_pct_change: skew 23.98, kurtosis 1159.8 — both inflated by the single +336.84% Mt. Gox print; rsi_14: skew 0.270 (symmetric). Mean absolute daily move by era: 4.245%, 3.433%, 2.724%, 2.210%, 1.822%.

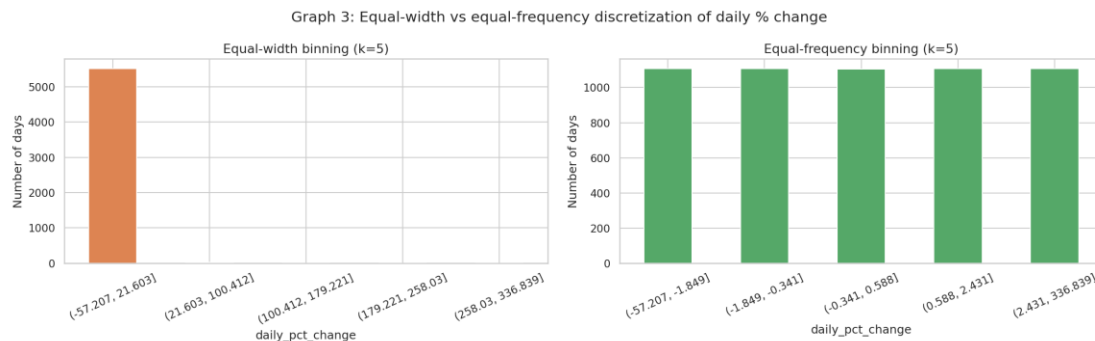
Null handling. 279 nulls in 7 columns over 199 rows. Structural (rolling-window) nulls are the burn-in and were dropped. A recovery test — 200 values removed at random and re-imputed — ranked the strategies:

Strategy	MAE	RMSE
Mean imputation	11.118	13.716
Median imputation	10.957	13.706
Forward fill	3.175	4.523
Linear interpolation	2.483	3.416

Normalization. Between two sample days the raw Euclidean distance is 43,056.6, of which the close price contributes 100.00%; after z-scoring the distance is 2.275 and all attributes contribute.

Discretization. With $k = 5$ on `daily_pct_change`, equal-width binning puts 99.6% of all days into one bin (the range is stretched by the outlier), while equal-frequency keeps every bin near 20%.

Domain bins produced the categorical attributes used later: `move_class`, `rsi_zone` (3.78% Oversold / 81.10% Neutral / 15.12% Overbought), `range_band` and `trend` (63.84% above the 200-day MA).



Equal-width vs equal-frequency binning of daily % change.

4. Conclusion

The cleaning pipeline was corrected (it previously failed outright on its configured input), and the four preprocessing tasks were carried out on 5,534 cleaned records. Descriptive statistics quantified the skew and the fall in volatility across halving eras; nulls were separated into structural and at-random and treated accordingly, with interpolation confirmed as the most accurate filler; three normalization methods were implemented and shown to be necessary for distance-based mining; and three discretization methods were implemented, with equal-width shown to collapse on skewed financial data. The result, `btc_preprocessed_v1.csv`, is the input for Experiments 5, 7 and 8.

EXPERIMENT NO. 5

Naive Bayesian and Classification Tree Algorithms

Next-day direction prediction on the Bitcoin dataset, with an honest benchmark

1. Aim

To implement the Naive Bayesian classifier and a classification (decision) tree, apply them to the Bitcoin case study to predict the next day's direction, and evaluate them with a confusion matrix and the standard classification metrics.

2. Theory

2.1 Classification

Classification is supervised learning: a model is induced from labelled training data and used to assign a class label to unseen records. It has two steps — **model construction** (learning) and **model usage** (prediction), with accuracy estimated on a test set that the model has not seen.

2.2 Naive Bayes classifier

A statistical classifier based on **Bayes' theorem**:

$$P(C_k | X) = P(X | C_k) \times P(C_k) / P(X)$$

where $P(C_k|X)$ is the **posterior**, $P(X|C_k)$ the **likelihood**, $P(C_k)$ the **prior** and $P(X)$ the **evidence**. Since $P(X)$ is the same for every class it is dropped from the comparison.

The **naive** assumption is **class-conditional independence** of the attributes:

$$P(X | C_k) = \prod_{i=1..n} P(x_i | C_k) \\ \Rightarrow \text{predict } \operatorname{argmax}_k P(C_k) \prod P(x_i | C_k)$$

For a **continuous** attribute the likelihood is taken as a Gaussian:

$$P(x_i | C_k) = (1 / \sqrt{2\pi\sigma^2}) \times \exp(- (x_i - \mu)^2 / (2\sigma^2))$$

For a **categorical** attribute it is a counted frequency, with **Laplace correction** (add 1 to every count) so that an unseen value does not force the whole product to zero. Because the product of many small probabilities underflows, implementations sum logarithms instead.

Advantages: fast, needs little training data, handles many attributes, works well in practice even when the independence assumption is violated. **Limitations:** the independence assumption is usually false; correlated attributes are effectively double-counted; it cannot model interactions.

2.3 Classification tree

A **decision tree** is a flowchart: each internal node tests an attribute, each branch is an outcome, each leaf holds a class label. Trees are induced top-down by recursively choosing the attribute that best separates the classes (ID3, C4.5, CART).

Entropy	$H(S) = - \sum_k p_k \log_2 p_k$	
Information gain	$IG(S, A) = H(S) - \sum_v (S_v / S) \times H(S_v)$	
Gini index	$Gini(S) = 1 - \sum_k p_k^2$	
Gain ratio	$GainRatio(A) = IG(S, A) / SplitInfo(A)$	[C4.5, corrects ID3's bias towards many-valued attributes]

Stopping criteria: all records in a node share a class, no attribute gives positive gain, the node is too small, or the maximum depth is reached. **Pruning** (pre- or post-) controls over-fitting.

Advantages: readable rules, no scaling needed, handles mixed attribute types. **Limitations:** overfits easily, unstable (a small data change can change the tree), and greedy splitting gives no global optimality guarantee.

2.4 Evaluation

Confusion matrix	Predicted Positive	Predicted Negative
------------------	--------------------	--------------------

Actual Positive	TP	FN
------------------------	----	----

Actual Negative	FP	TN
------------------------	----	----

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$

Precision = $TP / (TP + FP)$ Recall (sensitivity) = $TP / (TP + FN)$

Specificity = $TN / (TN + FP)$ F1 = $2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$

ROC-AUC = area under the TPR-vs-FPR curve; 0.5 = random, 1.0 = perfect

For a time series the test set must follow the training set in time. A random split leaks future information into training and inflates every metric.

3. Procedure

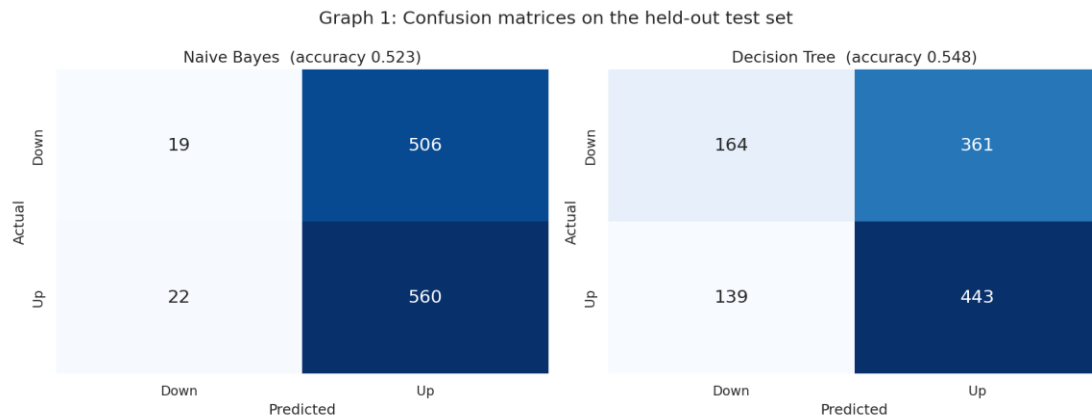
1. Build the label: Up if tomorrow's close exceeds today's, else Down (2,909 Up / 2,624 Down).
2. Build 11 features from data available at today's close; drop rows lost to the lag features (5,531 usable rows).
3. Split chronologically 80/20 — train 4,424 rows (to 15 Mar 2023), test 1,107 rows.
4. Fit Naive Bayes: estimate $P(C_k)$, and μ, σ^2 per feature per class; score with log-probabilities.
5. Induce the tree: search (feature, threshold) pairs for the largest information gain, recurse to depth 4 with a minimum leaf size of 25.
6. Predict on the test set, build the confusion matrices and compute every metric by hand.
7. Verify against scikit-learn; check sensitivity to the split point and to tree depth.

4. Results

Model	Accuracy	Precision	Recall	Specificity	F1	AUC
Naive Bayes	0.5230	0.5253	0.9622	0.0362	0.6796	0.5075
Decision Tree (d=4)	0.5483	0.5510	0.7612	0.3124	0.6392	0.5462
Majority baseline	0.5257	0.5257	1.0000	0.0000	0.6892	0.5000

Confusion matrix	Pred. Down	Pred. Up
Naive Bayes — actual Down	19	506
Naive Bayes — actual Up	22	560
Tree — actual Down	164	361
Tree — actual Up	139	443

Root of the induced tree: $\text{rsi}_{14} \leq 57.17$, information gain 0.0059 bits out of a root entropy of 0.9981 bits. **Verification:** the from-scratch Naive Bayes matches scikit-learn on 100.00% of test predictions.



Confusion matrices of the two classifiers on the held-out test set.

5. Conclusion

Both algorithms were implemented from first principles and verified against a library implementation. The decision tree reached 54.83% accuracy (AUC 0.546) against a 52.57% majority-class baseline, while Naive Bayes reached 52.30% (AUC 0.508) — no better than always predicting "Up", because it predicts Up on 96% of test days.

The limiting factor is the data, not the code: the root entropy of 0.9981 bits is reduced by only 0.0054 bits (0.54%) by the best available feature, and the Gaussian class densities for Up and Down days almost coincide. Next-day direction in a liquid market is close to a fair coin, so a result near the baseline is the expected and honest outcome; the experiment's value is a correct implementation and a documented benchmark that any future model must beat.

EXPERIMENT NO. 6

Implementation of Clustering Algorithm (K-Means)

Case study: Predicting Trends and Uncovering Patterns in Bitcoin Price Movement

1. Aim

To study the K-Means clustering algorithm, solve a numerical example by hand, and implement the algorithm without using any inbuilt clustering library function on the Bitcoin case-study dataset.

2. Objectives

- To understand clustering as an unsupervised data-mining task.
- To study the K-Means algorithm, its distance measure and its objective function.
- To solve a two-dimensional K-Means problem manually, iteration by iteration.
- To group Bitcoin trading days into market regimes and evaluate the clustering.

3. Theory

3.1 Clustering

Clustering is the process of partitioning a set of data objects into groups (clusters) such that objects within a cluster are similar to one another and dissimilar to objects in other clusters. It is an **unsupervised** technique — no class labels are given, and the grouping is discovered from the data itself. Similarity between two numeric objects is measured using a distance function, most commonly the Euclidean distance.

Types of clustering methods:

- **Partitional** — divides data into k non-overlapping clusters (K-Means, K-Medoids).
- **Hierarchical** — builds a tree of clusters, agglomerative or divisive (AGNES, DIANA).
- **Density-based** — grows clusters in dense regions and marks sparse points as noise (DBSCAN).
- **Grid-based** — quantises the space into cells and clusters the cells (STING, CLIQUE).

3.2 K-Means algorithm

K-Means is a partitional, centroid-based clustering algorithm proposed by MacQueen (1967). It divides n objects into k clusters, where each cluster is represented by the mean (centroid) of the objects in it. The algorithm minimises the **Sum of Squared Errors (SSE)**, also called inertia:

$$SSE = \sum_{i=1}^k \sum_{p \in C_i} ||p - c_i||^2$$

The distance between a point p and a centroid c in two dimensions is:

$$d(p, c) = \sqrt{(p_x - c_x)^2 + (p_y - c_y)^2}$$

A centroid is recomputed as the arithmetic mean of its members:

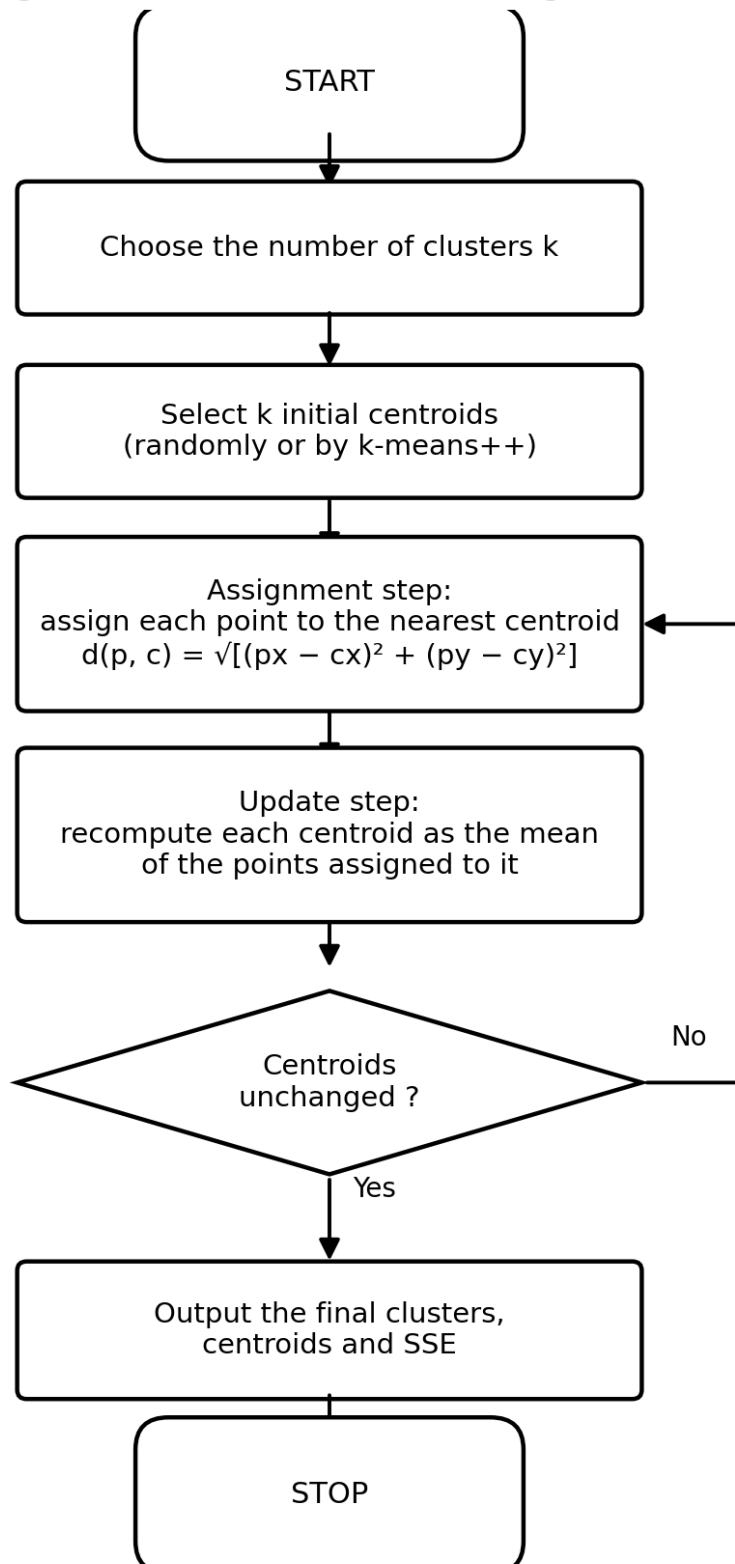
$$c_i = (1 / |C_i|) \sum_{p \in C_i} p$$

Steps:

1. Choose the number of clusters k.
2. Select k initial centroids (randomly, or spread out using k-means++).
3. **Assignment step:** assign every object to the cluster whose centroid is nearest to it.

4. **Update step:** recompute each centroid as the mean of the objects assigned to it.
5. Repeat steps 3 and 4 until the centroids no longer change (or a maximum number of iterations is reached). SSE decreases at every step, so the algorithm always terminates.

Fig. 1 Flowchart of the K-Means algorithm



3.3 Characteristics of K-Means

Advantages

Limitations

Advantages	Limitations
Simple to understand and easy to implement	The number of clusters k must be decided in advance
Efficient: time complexity $O(n k d i)$ for n objects, k clusters, d dimensions and i iterations	The result depends on the initial centroids — only a local optimum is guaranteed
Scales well to large datasets	Assumes clusters are roughly spherical and of similar size
Produces tight, well-separated clusters when the data suits it	The mean is highly sensitive to outliers and noise
Cluster centroids are directly interpretable	Not suitable for categorical attributes (K-Modes is used instead)

3.4 Applications

Customer segmentation, market-regime detection in financial time series, document and image segmentation, anomaly and fraud detection, and vector quantisation for data compression.

4. Solved Numerical Example

Problem. Eight Bitcoin trading days are described by two attributes taken from the cleaned dataset:

- x = daily percentage change = $(\text{close} - \text{open}) / \text{open} \times 100$
- y = intraday range as a percentage of the open = $(\text{high} - \text{low}) / \text{open} \times 100$

Cluster these eight days into $k = 2$ clusters using K-Means, taking points **A** and **C** as the initial centroids. (All values are rounded to one decimal place; distances to two.)

Point	Date	x = daily % change	y = range % of open
A	2022-01-18	1.0	3.3
B	2025-03-01	2.3	3.2
C	2019-09-10	-1.8	4.1
D	2017-12-17	-1.5	5.8
E	2013-11-29	9.6	17.4
F	2018-02-06	11.8	28.6
G	2013-12-07	-17.5	37.9
H	2020-03-12	-27.2	30.1

Initial centroids: $C1 = A = (1.0, 3.3)$ $C2 = C = (-1.8, 4.1)$

Iteration 1

Distance of every point from both centroids, e.g. for point E:

$$d(E, C1) = \sqrt{(9.6 - 1.0)^2 + (17.4 - 3.3)^2} = \sqrt{73.96 + 198.81} = \sqrt{272.77} = 16.52$$

$$d(E, C2) = \sqrt{(9.6 + 1.8)^2 + (17.4 - 4.1)^2} = \sqrt{129.96 + 176.89} = \sqrt{306.85} = 17.52$$

Point	(x, y)	$d(C1) = (1.0, 3.3)$	$d(C2) = (-1.8, 4.1)$	Nearest	Cluster
-------	----------	----------------------	-----------------------	---------	---------

Point	(x, y)	d(C1) = (1.0, 3.3)	d(C2) = (-1.8, 4.1)	Nearest	Cluster
A	(1.0, 3.3)	0.00	2.91	C1	1
B	(2.3, 3.2)	1.30	4.20	C1	1
C	(-1.8, 4.1)	2.91	0.00	C2	2
D	(-1.5, 5.8)	3.54	1.73	C2	2
E	(9.6, 17.4)	16.52	17.52	C1	1
F	(11.8, 28.6)	27.51	28.02	C1	1
G	(-17.5, 37.9)	39.24	37.27	C2	2
H	(-27.2, 30.1)	38.90	36.35	C2	2

Clusters: C1 = {A, B, E, F} C2 = {C, D, G, H} **SSE = 3744.27**

New centroids:

$$C1 = ((1.0 + 2.3 + 9.6 + 11.8)/4 , (3.3 + 3.2 + 17.4 + 28.6)/4) = (24.7/4 , 52.5/4) = (6.17, 13.12)$$

$$C2 = ((-1.8 - 1.5 - 17.5 - 27.2)/4 , (4.1 + 5.8 + 37.9 + 30.1)/4) = (-48.0/4 , 77.9/4) = (-12.00, 19.48)$$

Iteration 2

Point	(x, y)	d(C1) = (6.17, 13.12)	d(C2) = (-12.00, 19.48)	Nearest	Cluster
A	(1.0, 3.3)	11.10	20.76	C1	1
B	(2.3, 3.2)	10.65	21.67	C1	1
C	(-1.8, 4.1)	12.04	18.45	C1	1 ← moved
D	(-1.5, 5.8)	10.60	17.25	C1	1 ← moved
E	(9.6, 17.4)	5.48	21.70	C1	1
F	(11.8, 28.6)	16.47	25.49	C1	1
G	(-17.5, 37.9)	34.27	19.22	C2	2
H	(-27.2, 30.1)	37.44	18.54	C2	2

Clusters: C1 = {A, B, C, D, E, F} C2 = {G, H} **SSE = 1508.62**

Points C and D have moved from cluster 2 to cluster 1 — this is the reassignment that K-Means performs as the centroids shift.

New centroids:

$$C1 = (21.4/6 , 62.4/6) = (3.57, 10.40)$$

$$C2 = ((-17.5 - 27.2)/2 , (37.9 + 30.1)/2) = (-22.35, 34.00)$$

Iteration 3

Point	(x, y)	d(C1) = (3.57, 10.40)	d(C2) = (-22.35, 34.00)	Nearest	Cluster
A	(1.0, 3.3)	7.55	38.57	C1	1
B	(2.3, 3.2)	7.31	39.45	C1	1
C	(-1.8, 4.1)	8.28	36.28	C1	1
D	(-1.5, 5.8)	6.85	35.07	C1	1

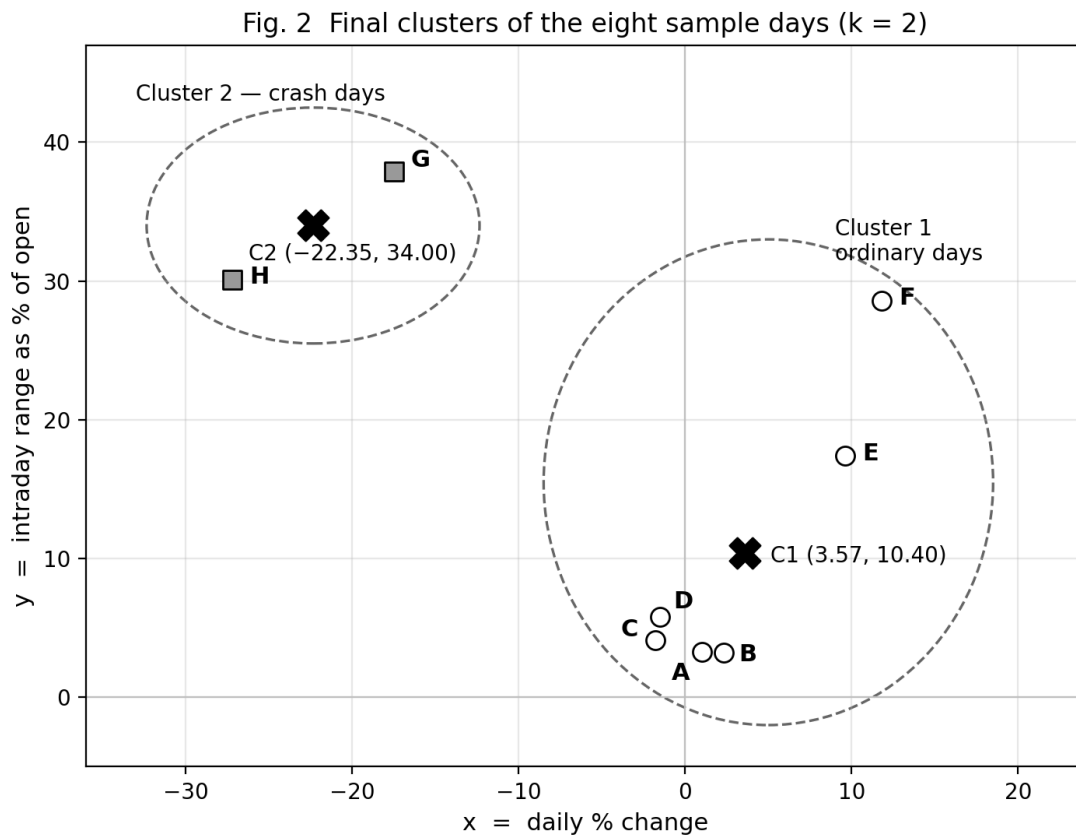
Point	(x, y)	d(C1) = (3.57, 10.40)	d(C2) = (-22.35, 34.00)	Nearest	Cluster
E	(9.6, 17.4)	9.24	36.01	C1	1
F	(11.8, 28.6)	19.97	34.57	C1	1
G	(-17.5, 37.9)	34.64	6.22	C2	2
H	(-27.2, 30.1)	36.54	6.22	C2	2

The membership is unchanged, so the recomputed centroids are the same as the previous ones: $C1 = (3.57, 10.40)$, $C2 = (-22.35, 34.00)$. **The algorithm has converged after 3 iterations.**

Final answer

Cluster	Members	Final centroid	Meaning
1	A, B, C, D, E, F	(3.57, 10.40)	ordinary trading days — small % change, narrow intraday range
2	G, H	(-22.35, 34.00)	crash days — large negative change, very wide range

SSE falls monotonically: **3744.27** → **1508.62** → **787.66**, which is the behaviour the theory predicts. The two days isolated in cluster 2 are 07 Dec 2013 (the end of the first big bubble) and 12 Mar 2020 (“Black Thursday”, the COVID crash).



Effect of the initial centroids

Re-running the same eight points with different seeds shows that K-Means only guarantees a local optimum:

Initial centroids	Iterations	Final clusters	SSE
A, C	3	{A B C D E F} {G H}	787.66
A, G	2	{A B C D E F} {G H}	787.66
B, E	3	{A B C D E} {F G H}	1105.75

The seeds B, E converge to a different, worse partition. This is why K-Means is normally restarted several times and the run with the lowest SSE is kept.

5. Implementation Summary (soft copy)

The same algorithm was implemented in Python without any clustering library and applied to all 5,534 cleaned Bitcoin trading days (Feb 2011 – Mar 2026), using the same two features. 72 shock days flagged as outliers in Experiment 5 ($|z| > 3$) were separated first, because a single day (26 Feb 2014: +336.8% change, 378.7% range) otherwise consumed an entire cluster. The remaining 5,462 days were clustered with $k = 4$, chosen from the elbow curve and the requirement of the case study.

Cluster	Regime	Centroid (x, y)	Days	Share
C1	Calm / Sideways	(−0.48, 3.28)	3652	66.9%
C2	Mild Bullish Drift	(3.46, 6.64)	1097	20.1%
C3	Volatile Bearish (Sell-off)	(−6.68, 13.00)	462	8.5%
C0	Volatile Bullish (Rally)	(9.58, 15.71)	251	4.6%

Performance matrix: SSE = 3160.65 | mean silhouette = 0.4746 | Davies–Bouldin index = 0.7487 | Calinski–Harabasz index = 4468.75 | converged in 12 iterations (best of 10 restarts).

6. Conclusion

In this experiment the K-Means clustering algorithm was studied and implemented from first principles. A two-dimensional numerical example of eight Bitcoin trading days was solved by hand with $k = 2$; the algorithm converged in three iterations to clusters {A, B, C, D, E, F} and {G, H} with centroids (3.57, 10.40) and (−22.35, 34.00) and a final SSE of 787.66, and the program output matched the manual calculation exactly.

The same implementation applied to 5,462 Bitcoin trading days produced four interpretable market regimes — calm, mild bullish drift, volatile sell-off and volatile rally — showing that roughly 87% of Bitcoin's days are low-volatility days and that the share of turbulent days has fallen in every successive halving era. The experiment also demonstrated K-Means' sensitivity to outliers and to the choice of initial centroids, both of which had to be handled explicitly to obtain a usable clustering.

EXPERIMENT NO. 7

Implementation of Hierarchical Clustering

Agglomerative clustering of Bitcoin market periods, with dendrograms and cluster interpretation

1. Aim

To implement an agglomerative hierarchical clustering method on the Bitcoin dataset, generate a dendrogram, and analyse the hierarchical relationships among the data instances.

2. Theory

2.1 Hierarchical clustering

Hierarchical clustering builds a nested sequence of partitions instead of a single flat one. The number of clusters need not be given in advance: the tree is built once and cut afterwards at whatever level is useful. Two families exist:

- **Agglomerative (AGNES)** — *bottom-up*. Every object starts as its own cluster; the two closest clusters are merged repeatedly until one remains.
- **Divisive (DIANA)** — *top-down*. All objects start in one cluster, which is split repeatedly until every object is alone.

The result is drawn as a **dendrogram**: a tree whose leaves are the objects and whose merge heights are the distances at which clusters joined. Cutting the dendrogram with a horizontal line at height h yields a flat clustering.

2.2 The AGNES algorithm

1. Treat each of the n objects as a cluster; compute the $n \times n$ **proximity matrix**.
2. Find the closest pair of clusters and **merge** them.
3. **Update** the distances from the merged cluster to all others using the linkage rule.
4. Repeat steps 2-3 until a single cluster remains, recording every merge and its height.

Complexity: $O(n^2)$ memory for the proximity matrix and $O(n^3)$ time in the naive form, which is why hierarchical clustering is applied to hundreds of objects, not millions.

2.3 Linkage criteria

Linkage	Distance between clusters A and B	Behaviour
Single (MIN)	$\min d(a, b)$	finds elongated clusters; suffers from chaining
Complete (MAX)	$\max d(a, b)$	compact clusters; sensitive to outliers
Average (UPGMA)	mean of all $d(a, b)$	a compromise; usually preserves distances best
Ward	merge that increases within-cluster variance least	balanced, similar-sized clusters

All four are special cases of the **Lance-Williams** update:

$$d(k, i \cup j) = \alpha_i d(k, i) + \alpha_j d(k, j) + \beta d(i, j) + \gamma |d(k, i) - d(k, j)|$$

Single : $\alpha_i = \alpha_j = 1/2$, $\beta = 0$, $\gamma = -1/2$
 Complete: $\alpha_i = \alpha_j = 1/2$, $\beta = 0$, $\gamma = +1/2$
 Average : $\alpha_i = n_i/(n_i+n_j)$, $\alpha_j = n_j/(n_i+n_j)$, $\beta = \gamma = 0$
 Ward : $\alpha_i = (n_i+n_k)/(n_i+n_j+n_k)$, $\alpha_j = (n_j+n_k)/(n_i+n_j+n_k)$,
 $\beta = -n_k/(n_i+n_j+n_k)$, $\gamma = 0$ [on squared distances]

2.4 Evaluating the tree

The **cophenetic correlation coefficient** is the correlation between the original pairwise distances and the heights at which each pair is first joined in the dendrogram. A value near 1 means the tree distorts the data very little.

3. Procedure

1. Aggregate the cleaned daily data into **monthly market profiles** (mean return, volatility, mean intraday range, % up days) and standardise them.
2. Exclude 2 months rebuilt by interpolation and 4 statistical outlier months, leaving **176**.
3. Build the proximity matrix and run AGNES with each of the four linkage rules.
4. Compare the rules on cophenetic correlation and on the balance of the $k = 4$ cut; choose Ward.
5. Cut the dendrogram at $k = 4$ and profile each cluster in the original units.
6. Repeat on a 500-day sample in the Experiment 6 feature space and compare with K-Means.

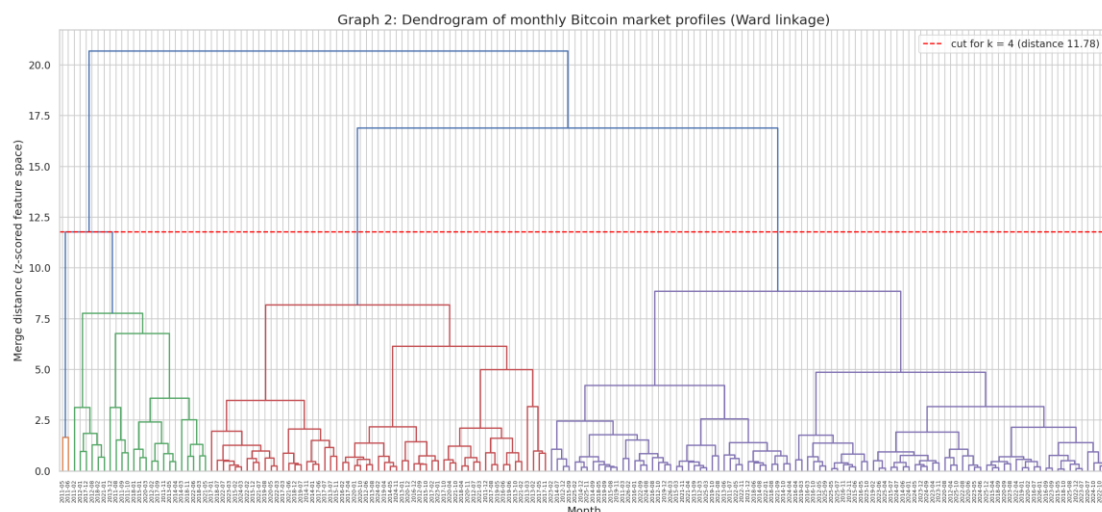
4. Results

Verification: the from-scratch implementation reproduces SciPy's merge sequence exactly, with merge heights agreeing to 1.8×10^{-15} .

Linkage	Cophenetic corr.	Sizes at $k = 4$	Largest cluster
Single	0.8274	172, 2, 1, 1	97.7%
Complete	0.7387	160, 10, 4, 2	90.9%
Average	0.8483	168, 4, 3, 1	95.5%
Ward (chosen)	0.6063	94, 57, 23, 2	53.4%

The four monthly market types (Ward, $k = 4$):

Cluster	Months	Mean return	Volatility	Mean range	% up days	Reading
4	94	-0.07%	2.47	3.78%	48.0%	Quiet drift
3	57	+0.90%	3.72	5.81%	61.5%	Steady bull month
2	23	-0.27%	7.27	10.57%	45.7%	Turbulent / bear month
1	2	+3.46%	14.19	18.38%	50.8%	Mania (May-Jun 2011)



Dendrogram of 176 monthly market profiles (Ward linkage); the dashed line is the $k = 4$ cut.

Cross-check with K-Means (Experiment 6): on the same 500 trading days the two partitions agree on **92.2%** of days, Adjusted Rand Index **0.7895**.

5. Conclusion

Agglomerative hierarchical clustering was implemented from first principles, including the Lance-Williams update for four linkage rules, and verified against SciPy. Average linkage preserved distances best but produced a useless 95.5%/4.5% cut; Ward linkage gave the only balanced partition and four interpretable month types. The dendrogram separates volatility before direction, and shows quiet months displacing turbulent ones over the years — no turbulent month since 2022 against five in 2011. On individual days the hierarchy agrees with the K-Means regimes on 92.2% of cases (ARI 0.79), confirming that the regime structure is a property of the data rather than of either algorithm.

EXPERIMENT NO. 8

Implementation of Association Rule Mining (Apriori)

Frequent itemsets and association rules from Bitcoin trading days

1. Aim

To implement the Apriori association-rule-mining algorithm on the Bitcoin dataset, generate frequent itemsets and association rules, and evaluate them using support, confidence and lift.

2. Theory

2.1 Association rule mining

Association rule mining finds relationships of the form $X \rightarrow Y$ among items in a transaction database, where X (the **antecedent**) and Y (the **consequent**) are disjoint itemsets. Its classical application is **market-basket analysis** ("customers who buy bread and butter also buy milk"). Here each trading day is a basket and its discretized attributes are the items.

2.2 Interest measures

$\text{support}(X) = \text{count}(X) / N$ [how often the pattern occurs]
 $\text{confidence}(X \rightarrow Y) = \text{support}(X \cup Y) / \text{support}(X)$ [reliability of the rule]
 $\text{lift}(X \rightarrow Y) = \text{confidence}(X \rightarrow Y) / \text{support}(Y)$ [1 = independent, >1 positive, <1 negative]
 $\text{leverage}(X \rightarrow Y) = \text{support}(X \cup Y) - \text{support}(X) \cdot \text{support}(Y)$
 $\text{conviction}(X \rightarrow Y) = (1 - \text{support}(Y)) / (1 - \text{confidence})$

Confidence alone is misleading when the consequent is common: a rule can have high confidence simply because Y occurs on most days. **Lift** corrects for that, which is why rules are ranked by lift.

2.3 The Apriori property

Downward closure: every non-empty subset of a frequent itemset is itself frequent.

Contrapositive: if an itemset is infrequent, every superset of it is infrequent too. This single property lets the algorithm discard entire branches of the search space without counting them.

2.4 The algorithm

1. Scan the database and find the frequent 1-itemsets L_1 .
2. **Join:** form candidates C_k from pairs of L_{k-1} that share their first $k-2$ items.
3. **Prune:** discard any candidate having an infrequent $(k-1)$ -subset (the Apriori property).
4. **Count:** one scan of the database; keep candidates with $\text{support} \geq \text{min_support} \rightarrow L_k$.
5. Repeat with $k = k+1$ until L_k is empty.
6. **Generate rules:** for each frequent itemset, try every non-empty proper subset as the antecedent and keep the rules meeting min_confidence .

Limitation: Apriori scans the database once per level and can generate very many candidates; **FP-Growth** avoids candidate generation by compressing the database into an FP-tree.

3. Procedure

1. Convert each of 5,533 trading days into a transaction of 7 items (MOVE, RANGE, RSI, TREND, MACD, VOL, NEXT) using the discretization of Experiment 4 — 20 distinct items in all.
2. Run Apriori at $\text{min_support} = 0.05$ (276 days).

3. Generate rules at min_confidence = 0.60 and compute lift, leverage and conviction.
4. Verify against exhaustive enumeration for all itemsets of size ≤ 3 .
5. Repeat the run across seven support thresholds to observe the cost/benefit trade-off.

4. Results

k	Candidates joined	Pruned	Counted	Frequent
1	20	—	20	19
2	171	0	171	112
3	484	132	352	265
4	564	231	333	200
5	185	125	60	32
6	7	5	2	1

629 frequent itemsets (largest of size 6) and **737 rules**; **493 candidates** were pruned without ever being counted. Verification against exhaustive enumeration: 396 itemsets of size ≤ 3 found by both methods, identical sets, support difference 0.00.

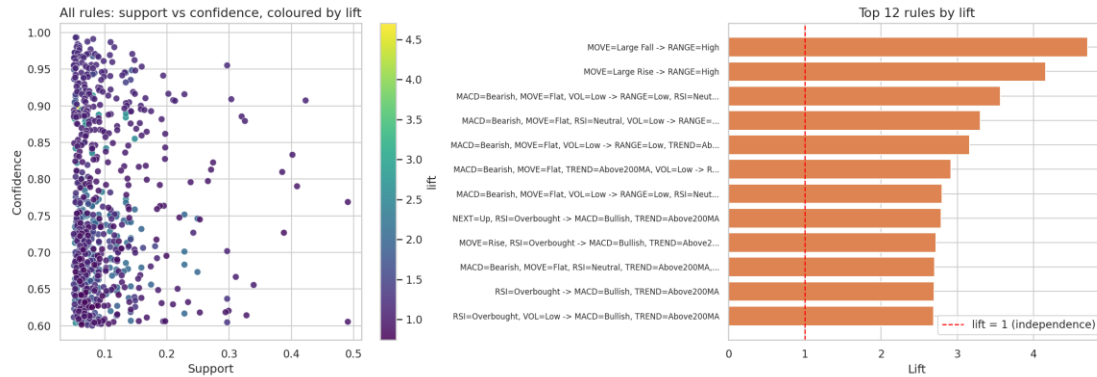
Strongest rules (same-day structure):

Rule	Support	Confidence	Lift
{MOVE=Large Fall} $\rightarrow$ {RANGE=High}	0.058	0.896	4.71
{MOVE=Large Rise} $\rightarrow$ {RANGE=High}	0.065	0.792	4.16
{MACD=Bearish, MOVE=Flat, VOL=Low} $\rightarrow$ {RANGE=Low, RSI=Neutral, TREND=Above200MA}	0.053	0.682	3.56
{RSI=Overbought} $\rightarrow$ {MACD=Bullish, TREND=Above200MA}	0.134	0.884	2.70

Best predictive rule: {MACD=Bullish, RSI=Overbought, TREND=Above200MA} $\rightarrow$ {NEXT=Up}, support 0.081 (447 days), confidence 0.604, lift 1.167 — against a base rate $P(\text{NEXT=Up}) = 0.518$. It is the only pure next-day rule above 0.60 confidence, and no rule in the whole search exceeds a lift of 1.17.

min_support	Itemsets	Rules	Runtime
0.02	1,579	2,154	1.25 s
0.05	629	737	0.52 s
0.12	176	155	0.18 s
0.30	25	13	0.06 s

Graph 2: Association rules discovered by Apriori



Left: every rule in the support-confidence plane, coloured by lift. Right: the twelve highest-lift rules — all of them same-day associations.

5. Conclusion

Apriori was implemented from first principles and verified against exhaustive enumeration. On 5,533 daily transactions it produced 629 frequent itemsets and 737 rules, with the Apriori property pruning 493 candidates before they were ever counted.

The rules divide in two. **Same-day associations are strong:** a large move implies a wide intraday range (lift 4.71) and the momentum indicators imply one another (lift 2.70) — the algorithm independently rediscovering that RSI, MACD and the 200-day trend are functions of the same price series. **Next-day associations are weak:** the single best rule raises the probability of an up day from 51.8% to 60.4% on 8% of days, and nothing in the search exceeds a lift of 1.17. That matches the classification results of Experiment 5 and is the honest finding for a liquid market.